

INFORMATION RETRIEVAL

Probabilistic IR

Corso di Laurea Magistrale in Informatica

Università di Roma Tor Vergata

Prof. Giorgio Gambosi, Prof. Danilo Croce

a.a. 2025-2026

Derived from slides originally produced by C. Manning and by H. Schütze



Probabilistic Approach to Retrieval

- ⊙ Given a user information need (represented as a query) and a collection of documents (transformed into document representations), a system must determine how well the documents satisfy the query
 - An IR system has an **uncertain understanding** of the user query, and makes an **uncertain guess** of whether a document satisfies the query
- ⊙ Probability theory provides a principled foundation for such **reasoning under uncertainty**
 - Probabilistic models exploit this foundation to estimate how likely it is that a document is relevant to a query

Why probabilistic?

At first glance, a document d is either relevant or nonrelevant with respect to a query q .

Indeed there are several sources of uncertainty:

- ⊙ across different users: different users may have different opinions regarding the relevance of d with respect to q
- ⊙ for the same user in different contexts: a user may judge d relevant or not in dependence of many factors, in different contexts
- ⊙ with respect to the document representation: d is usually represented in some way in the IR system; hence relevance is estimated on limited information
- ⊙ with respect to the IR system: the system itself may induce approximations/errors in estimating the relevance of d

In a probabilistic model, documents are retrieved/ranked by their (estimated) probability of being relevant, given the query

$$p(R = 1 \mid d, q)$$

- ⊙ Classical probabilistic retrieval model
 - Probability ranking principle
 - Binary Independence Model, BestMatch25 (Okapi)
- ⊙ Language model approach to IR
- ⊙ ...

Probabilistic methods are one of the oldest but also one of the hottest topics in IR

- ⊙ Vector space model: rank documents according to similarity to query.
- ⊙ The notion of similarity does not translate directly into an assessment of “is the document a good document to give to the user or not?”
 - The most similar document can be highly relevant or completely nonrelevant.
- ⊙ Probability theory is arguably a cleaner formalization of what we want an IR system to do: rank documents by their estimated probability of relevance.

Basic Probability Theory

- ⊙ For events A and B
 - Joint probability $p(A \cap B)$ of both events occurring
 - Conditional probability $p(A|B)$ of event A occurring given that event B has occurred
- ⊙ **Chain rule** gives fundamental relationship between joint and conditional probabilities:

$$p(A, B) = p(A \cap B) = p(A|B) \cdot p(B) = p(B|A) \cdot p(A)$$

- ⊙ Similarly for the complement of an event $p(\bar{A})$:

$$p(\bar{A}, B) = p(B|\bar{A}) \cdot p(\bar{A})$$

- ⊙ **Partition rule**: if B can be divided into an exhaustive set of disjoint subcases, then $p(B)$ is the sum of the probabilities of the subcases. A special case of this rule gives:

$$p(B) = p(A, B) + p(\bar{A}, B)$$

Basic Probability Theory

Bayes' Rule for inverting conditional probabilities:

$$p(A|B) = \frac{p(B|A) \cdot p(A)}{p(B)} = \left[\frac{p(B|A)}{\sum_{X \in \{A, \bar{A}\}} p(B|X) \cdot p(X)} \right] p(A)$$

Can be thought of as a way of updating probabilities:

- ⊙ Start off with **prior probability** $p(A)$ (initial estimate of how likely event A is in the absence of any other information)
- ⊙ Derive a **posterior probability** $p(A|B)$ after having seen the evidence B , based on the likelihood of B occurring in the two cases that A does or does not hold

Odds of an event provide a kind of multiplier for how probabilities change:

$$\text{Odds:} \quad O(A) = \frac{p(A)}{p(\bar{A})} = \frac{p(A)}{1 - p(A)}$$

- ⊙ Assume **binary** notion of relevance: $R_{d,q}$ is a **random** binary variable, such that
 - $R_{d,q} = 1$ if document d is relevant w.r.t query q
 - $R_{d,q} = 0$ otherwise
- ⊙ We may interpret $p(R_{d,q}) = p(R|d, q)$ as the probability that a **random user** judges d relevant for query q . In other words, assuming an event space defined on a set U of users (or user types), it is

$$p(R|d, q) = \sum_{u \in U} p(R|d, q, u)p(u)$$

where $p(R|d, q, u)$ is the probability that user u judges d relevant for query q , and $p(u)$ is the probability that u is the user asked to judge relevance

- ⊙ Documents should be ranked according to the **Bayes decision rule**: a document is preferred when

$$p(R \mid d, q) > p(\bar{R} \mid d, q)$$

or, equivalently, when

$$O(R \mid d, q) = \frac{p(R \mid d, q)}{p(\bar{R} \mid d, q)} > 1$$

Probabilistic relevance

$p(R|d, q)$ (and $p(\bar{R}|d, q)$) can be decomposed in two ways:



$$p(R|d, q) = \frac{p(d|R, q)p(R|q)}{p(d|q)} \quad p(\bar{R}|d, q) = \frac{p(d|\bar{R}, q)p(\bar{R}|q)}{p(d|q)}$$

that is, we look at the probability of relevant and not relevant documents when the query is fixed. This is the approach of BIM, 2-Poisson and BM25 models



$$p(R|d, q) = \frac{p(q|R, d)p(R|d)}{p(q|d)} \quad p(\bar{R}|d, q) = \frac{p(q|\bar{R}, d)p(\bar{R}|d)}{p(q|d)}$$

that is, we consider the probability of the query wrt to relevant and not relevant documents. This is the approach of language models

Probabilistic Ranking

- ⊙ Ranked retrieval setup: given a collection of documents, the user issues a query, and an ordered list of documents is returned
- ⊙ **Probabilistic ranking** orders documents decreasingly by their estimated probability of relevance w.r.t. query: $p(R_{d,q}) = p(R|d, q)$
- ⊙ in order to estimate and compare $p(R|d, q)$ and $p(R|d', q)$ several simplifying assumptions are done
 - **Independence assumption**: the relevance of each document is independent of the relevance of other documents

Let us consider the approach of considering

$$p(R|d, q) = \frac{p(d|R, q)p(R|q)}{p(d|q)} = \frac{p(d|R, q)p(R|q)}{p(d)}$$

where:

- ⊙ $p(d|R, q)$ is the probability that document d is randomly sampled from the subcollection of documents relevant for query q
- ⊙ $p(R|q)$ is the probability that a random document from the collection is relevant for q
- ⊙ $p(d|q) = p(d)$ (we assume d and q independent) is the probability that document d is sampled from the collection.

The same clearly holds for nonrelevant documents

$$p(\bar{R}|d, q) = \frac{p(d|\bar{R}, q)p(\bar{R}|q)}{p(d|q)} = \frac{p(d|\bar{R}, q)p(\bar{R}|q)}{p(d)}$$

moreover, either a document is relevant or it is nonrelevant, that is $p(\bar{R}|d, q) + p(R|d, q) = 1$

⊙ Assumptions:

- **uniform document probability:** $p(d) = p(d')$ for all d, d' (this could not be true if we consider document representations, but assume it holds, for the sake of simplicity)
- **$p(R|q)$ can be ignored:** if we are interested in ranking documents, the probabilities $p(R|q)$ and $p(\bar{R}|q)$ are constant on all documents, and can be ignored

Probability Ranking Principle (Robertson, 1977)

⊙ PRP in brief

- If the retrieved documents (w.r.t a query) are ranked decreasingly on their probability of relevance, then the effectiveness of the system will be the best that is obtainable

⊙ PRP in full

- If [the IR] system's response to each [query] is a ranking of the documents [...] in order of decreasing probability of relevance to the [query], **where the probabilities are estimated as accurately as possible on the basis of whatever data have been made available to the system for this purpose**, the overall effectiveness of the system to its user will be the best that is obtainable on the basis of those data

- ⊙ The overall goal of the IR system is to return the best possible top- k results in terms of relevance. How can we formalize “best”?
- ⊙ We may associate an error cost for each document wrongly returned (if not relevant) or not returned (if relevant)
- ⊙ The best result is then the one which minimizes the overall error cost
- ⊙ In a probabilistic framework the relevance of a document wrt a query q is a random variable with an associated probability $p(R|d, q)$: the error cost can only be defined in terms of expectation

- ⊙ Assume that the following costs are defined:
 - $C(d, q)$: the cost when d is a relevant document and it is not returned
 - $C'(d, q)$: the cost when d is a nonrelevant document and it is returned
- ⊙ If a set $D(q)$ of documents is returned by the system, the expected cost (**risk**) is given by

$$\begin{aligned} R(D(q)) &= \sum_{d \in D(q)} C'(d, q)p(\bar{R}|d, q) + \sum_{d \notin D(q)} C(d, q)p(R|d, q) \\ &= \sum_{d \in D(q)} C'(d, q)(1 - p(R|d, q)) + \sum_{d \notin D(q)} C(d, q)p(R|d, q) \end{aligned}$$

- ⊙ We may use the risk associated to a specific result as an inverse measure of the quality of the result

Probability Ranking Principle

Assume a simple binary error cost function is used, where $C(d, q) = C'(d, q) = 1$: that is, any case where a relevant document is not returned or a nonrelevant document is returned has a constant cost (say 1)

Then

$$R(D(q)) = \sum_{d \in D(q)} (1 - p(R|d, q)) + \sum_{d \notin D(q)} p(R|d, q) = |D(q)| + \sum_{d \notin D(q)} p(R|d, q) - \sum_{d \in D(q)} p(R|d, q)$$

If we assume a fixed $|D(q)| = k$, that is, the user is interested in the best k documents, then $R(D(q))$ is minimized if the set $D(q)$ is such that

$$\sum_{d \in D(q)} p(R|d, q) - \sum_{d \notin D(q)} p(R|d, q)$$

is maximized: this corresponds to $D(q)$ containing the k documents with highest probability of relevance $p(R|d, q)$.

Probability Ranking Principle

As a consequence of the above observations, the best retrieval policy is the one that, for any k , returns the k topmost documents in a ranking by non-increasing probability of relevance.

Theorem

When 0/1 loss is assumed, the PRP is optimal, in that it minimizes the (expected) loss

This clearly holds if all probabilities are correct

Why did we introduce loss?

- ⊙ The loss/risk formulation is used to **justify** the Probability Ranking Principle.
- ⊙ In particular, under **0/1 loss**, minimizing the expected retrieval error leads to the following rule:
 - rank documents by decreasing $p(R \mid d, q)$
- ⊙ After this justification, the loss no longer appears explicitly in the formulas:
 - it has been **absorbed into the derivation**
 - in the simple case considered here, its role is exhausted once PRP has been established
- ⊙ The next question is therefore not about loss anymore, but about estimation:
 - **how can we estimate $p(R \mid d, q)$ in practice?**
- ⊙ The Binary Independence Model (BIM) is one of the simplest answers to this question.

- ⊙ PRP tells us **how documents should be ranked**:
 - by decreasing probability of relevance $p(R \mid d, q)$
- ⊙ The practical problem is that these probabilities are **unknown**:
 - they must be estimated from the document representation and from collection statistics
- ⊙ The **Binary Independence Model** (BIM) is the simplest probabilistic model for doing this.
- ⊙ Main assumptions:
 - **Document independence**: the relevance of one document is independent of the relevance of the others
 - **Binary relevance**: a document is either relevant or nonrelevant
 - **Binary term representation**: each term is either present or absent in a document

Documents as sets of features

Documents can be represented for retrieval as vectors of feature values.

- ⊙ A document d is represented as a vector $(f_1, \dots, f_n)$.
- ⊙ Then relevance can be modeled as

$$p(R|f_1, \dots, f_n, q) = \frac{p(f_1, \dots, f_n|R, q)p(R|q)}{p(f_1, \dots, f_n|q)}$$
$$\stackrel{\text{rank}}{=} p(f_1, \dots, f_n|R, q)$$

- ⊙ Main assumption:
 - conditional feature independence

$$p(f_1, \dots, f_n|R, q) = \prod_i p(f_i|R, q)$$

- this is the Naive Bayes assumption

Binary Independence Model (BIM)

In BIM, each feature:

1. is associated to a term
2. is binary: 1 if the term occurs, 0 if it does not occur
3. document d represented by vector $v_d = (x_1, \dots, x_m)$, where $x_i = 1$ iff term t_i occurs in d
4. query q represented by vector $v_q = (y_1, \dots, y_m)$, where $y_i = 1$ iff term t_i occurs in q
5. different documents/queries may have the same vector representation

The conditional feature independence assumption can be interpreted as a no-association assumption between terms, conditioned on the query and on the relevance status of the document.

Binary incidence matrix

	Anthony and Cleopatra	Julius Caesar	The Tempest	Hamlet	Othello	Macbeth	...
Anthony	1	1	0	0	0	1	
Brutus	1	1	0	1	0	0	
Caesar	1	1	0	1	1	1	
Calpurnia	0	1	0	0	0	0	
Cleopatra	1	0	0	0	0	0	
mercy	1	0	1	1	1	1	
worser	1	0	1	1	1	0	
...							

Each document is represented as a **binary vector** $\in \{0, 1\}^{|V|}$.

To make probabilistic retrieval operational, we need a way to estimate how document terms contribute to relevance.

- ⊙ In BIM, documents and queries are represented through **binary term incidence**: a term is either present or absent.
- ⊙ We then estimate, for each query term, how informative its presence is for distinguishing relevant from nonrelevant documents.
- ⊙ This leads to an estimate of the probability of relevance $p(R \mid d, q)$, or equivalently of a ranking score that preserves the same ordering.

Binary Independence Model

In BIM, $p(R \mid d, q)$ is modeled using term incidence vectors as $p(R \mid v_d, v_q)$.

$$p(R \mid v_d, v_q) = \frac{p(v_d \mid R, v_q)p(R \mid v_q)}{p(v_d \mid v_q)}$$

$$p(\bar{R} \mid v_d, v_q) = \frac{p(v_d \mid \bar{R}, v_q)p(\bar{R} \mid v_q)}{p(v_d \mid v_q)}$$

- ⊙ $p(v_d \mid R, v_q)$ is the probability of observing representation v_d among relevant documents for query q .
- ⊙ $p(v_d \mid \bar{R}, v_q)$ is the probability of observing representation v_d among nonrelevant documents for query q .
- ⊙ These likelihoods will be compared through odds in order to derive a ranking function.

Deriving a Ranking Function for Query Terms (1)

- Given a query q , ranking documents by $p(R \mid d, q)$ is modeled under BIM as ranking them by $p(R \mid v_d, v_q)$.
- Instead of ranking by probability, we can equivalently rank by the odds of relevance:

$$\begin{aligned} O(R \mid v_d, v_q) &= \frac{p(R \mid v_d, v_q)}{p(\bar{R} \mid v_d, v_q)} \\ &= \frac{p(v_d \mid R, v_q)}{p(v_d \mid \bar{R}, v_q)} \cdot \frac{p(R \mid v_q)}{p(\bar{R} \mid v_q)} \end{aligned}$$

- For a fixed query, the factor $\frac{p(R \mid v_q)}{p(\bar{R} \mid v_q)}$ is constant across documents, and can be ignored for ranking:

$$O(R \mid v_d, v_q) \stackrel{\text{rank}}{=} \frac{p(v_d \mid R, v_q)}{p(v_d \mid \bar{R}, v_q)}$$

Deriving a Ranking Function for Query Terms (2)

By the **Naive Bayes conditional independence assumption**, the term indicators are conditionally independent given the query and the relevance status:

$$\frac{p(v_d|R, v_q)}{p(v_d|\bar{R}, v_q)} = \prod_{i=1}^M \frac{p(x_i|R, v_q)}{p(x_i|\bar{R}, v_q)}$$

Hence,

$$O(R|v_d, v_q)_{\text{rank}} = \prod_{i=1}^M \frac{p(x_i|R, v_q)}{p(x_i|\bar{R}, v_q)}$$

Deriving a Ranking Function for Query Terms (3)

Since each binary feature x_i can take only two values, either 0 or 1, the product can be split into two groups:

- ⊙ terms present in the document ($x_i = 1$)
- ⊙ terms absent from the document ($x_i = 0$)

Therefore,

$$O(R|v_d, v_q)^{\text{rank}} = \prod_{t_i: x_i=1} \frac{p(x_i = 1|R, v_q)}{p(x_i = 1|\bar{R}, v_q)} \cdot \prod_{t_i: x_i=0} \frac{p(x_i = 0|R, v_q)}{p(x_i = 0|\bar{R}, v_q)}$$

Deriving a Ranking Function for Query Terms

Additional simplifying assumption: terms not occurring in the query are equally likely to occur in relevant and nonrelevant documents (non-query-term assumption)

⊙ If $y_i = 0$, then $p(x_i = 1|R, v_q) = p(x_i = 1|\bar{R}, v_q)$

Hence, we obtain

$$O(R|v_d, v_q)^{\text{rank}} = \prod_{t_i: x_i=y_i=1} \frac{p(x_i = 1|R, v_q)}{p(x_i = 1|\bar{R}, v_q)} \cdot \prod_{t_i: x_i=0, y_i=1} \frac{p(x_i = 0|R, v_q)}{p(x_i = 0|\bar{R}, v_q)}$$

⊙ The left product is over query terms found in the document and the right product is over query terms not found in the document

Deriving a Ranking Function for Query Terms

- ⊙ Let $p_t = p(x_t = 1 | R, v_q)$ be the probability of a term appearing in a document relevant for q
- ⊙ Let $u_t = p(x_t = 1 | \bar{R}, v_q)$ be the probability of a term appearing in a document nonrelevant for q
- ⊙ This can be displayed as a contingency table:

	relevant document (R)	nonrelevant document ($\bar{R}$)
Term present ($x_t = 1$)	p_t	u_t
Term absent ($x_t = 0$)	$1 - p_t$	$1 - u_t$

Deriving a Ranking Function for Query Terms

All this results into

$$O(R|v_d, v_q)^{\text{rank}} = \prod_{t_i : x_i=y_i=1} \frac{p_i}{u_i} \cdot \prod_{t_i : x_i=0, y_i=1} \frac{1-p_i}{1-u_i}$$

By including the query terms found in the document into the **second product**, but simultaneously dividing by them in the **first product**, it results:

$$O(R|v_d, v_q)^{\text{rank}} = \prod_{t_i : x_i=y_i=1} \frac{p_i(1-u_i)}{u_i(1-p_i)} \cdot \prod_{t_i : y_i=1} \frac{1-p_i}{1-u_i}$$

Idea of the transformation

For every term with $x_i = y_i = 1$, we multiply by $1 = \frac{1-p_i}{1-u_i} \cdot \frac{1-u_i}{1-p_i}$. We put the factor $\frac{1-p_i}{1-u_i}$ into the second product, while the factor $\frac{1-u_i}{1-p_i}$ stays in the first product. Hence $\frac{p_i}{u_i} \cdot \frac{1-u_i}{1-p_i} = \frac{p_i(1-u_i)}{u_i(1-p_i)}$.

Deriving a Ranking Function for Query Terms

- ⊙ The **first product** is still over query terms found in the document, but the **right product** is now over all query terms, hence constant for a particular query and can be ignored.
- ⊙ The only value that needs to be estimated to rank documents w.r.t a query is the first product

$$O(R|v_d, v_q)^{\text{rank}} = \prod_{t_i : x_i=y_i=1} \frac{p_i(1-u_i)}{u_i(1-p_i)}$$

- ⊙ We can equally rank documents by the logarithm of this term, since log is a monotonic function. This is named **Retrieval Status Value** (RSV):

$$RSV_d = \log \prod_{t_i : x_i=y_i=1} \frac{p_i(1-u_i)}{u_i(1-p_i)} = \sum_{t_i : x_i=y_i=1} \log \frac{p_i(1-u_i)}{u_i(1-p_i)}$$

Deriving a Ranking Function for Query Terms

Equivalent: rank documents using the **log odds ratio** associated with each query term t_i :

$$c_i = \log \frac{p_i(1 - u_i)}{u_i(1 - p_i)} = \log \frac{p_i}{1 - p_i} - \log \frac{u_i}{1 - u_i}$$

⊙ c_i can be seen as the difference between two log-odds:

1. the log-odds $\log \frac{p_i}{1 - p_i}$ that term t_i is present in a relevant document
2. the log-odds $\log \frac{u_i}{1 - u_i}$ that term t_i is present in a nonrelevant document

⊙ Therefore, c_i is large when the term is much more likely in relevant than in nonrelevant documents.

Term weight c_i in BIM

$$c_i = \log \frac{p_i}{1 - p_i} - \log \frac{u_i}{1 - u_i}$$

can be seen as a term weight.

- ⊙ $c_i = 0$: term t_i has equal odds of appearing in relevant and nonrelevant documents
- ⊙ $c_i > 0$: term t_i has higher odds to appear in relevant documents
- ⊙ $c_i < 0$: term t_i has higher odds to appear in nonrelevant documents

Retrieval status value for document d :

$$RSV_d = \sum_{t_i : x_i=y_i=1} c_i$$

- ⊙ So BIM and vector space model are very similar on an operational level, except that the term weights are different.
- ⊙ In particular: we can use the same data structures (such as an inverted index) for the two models.

How to compute probability estimates

Which information can be used to compute the probabilities of a term t appearing in a relevant or nonrelevant document?

- ⊙ $p_i = p(x_i = 1 | R, v_q)$ probability that term x_i appears in a document relevant for q
- ⊙ $u_i = p(x_i = 1 | \bar{R}, v_q)$ probability that term x_i appears in a document nonrelevant for q

There are two possible scenarios:

- ⊙ There are some documents which we consider relevant and/or not relevant
 - A training set of relevance judgements given by users is available
 - Relevance judgements may derive by a **pseudo-relevance feedback** method
- ⊙ No information (relevance judgements) is available

How to compute probability estimates

First case: some relevance judgements are available (R is the number of documents judged relevant for the query).

For each term t_i in a query, estimate c_i in the whole collection using a contingency table of counts of documents in the collection, where:

- ⊙ N is the total number of documents
- ⊙ R is the number of relevant documents as derived from relevance judgements
- ⊙ r_i is the number of relevant documents in which term t_i occurs
- ⊙ df_{t_i} is the number of documents that contain term t_i

How to compute probability estimates

	Relevant documents	Nonrelevant documents	Total
Term present $x_i = 1$	r_i	$df_{t_i} - r_i$	df_{t_i}
Term absent $x_i = 0$	$R - r_i$	$(N - df_{t_i}) - (R - r_i)$	$N - df_{t_i}$
Total	R	$N - R$	N

Maximum-likelihood estimates:

$$p_i = p(x_i = 1 \mid R, q) \approx \frac{r_i}{R} \qquad u_i = p(x_i = 1 \mid \bar{R}, q) \approx \frac{df_{t_i} - r_i}{N - R}$$

So the BIM term weight becomes

$$c_i = \log \frac{p_i / (1 - p_i)}{u_i / (1 - u_i)} = \log \frac{\frac{r_i}{R - r_i}}{\frac{df_{t_i} - r_i}{(N - df_{t_i}) - (R - r_i)}}$$

Avoiding zeros: smoothing

- ⊙ If any of the counts is zero, the BIM term weight may be undefined.
- ⊙ Maximum-likelihood estimates are fragile for rare events.
- ⊙ To avoid this, we add a small constant α to each count, e.g. $\alpha = 0.5$.

This gives the smoothed weight

$$c_i \approx \log \frac{\frac{r_i + \alpha}{R - r_i + \alpha}}{\frac{\text{df}_{t_i} - r_i + \alpha}{(N - \text{df}_{t_i}) - (R - r_i) + \alpha}}$$

Exercise

- ⊙ Query: Obama health plan
- ⊙ Doc1: Obama rejects allegations about his own bad health
- ⊙ Doc2: The plan is to visit Obama
- ⊙ Doc3: Obama raises concerns with US health plan reforms

Estimate the quantities needed to rank the documents under BIM. Use a contingency table. Assume these are the only three documents in the collection.

Approximating nonrelevant documents

- ⊙ Suppose that relevant documents are only a small fraction of the collection.
- ⊙ Then, for estimating statistics of nonrelevant documents, we can approximate the nonrelevant set with the whole collection.
- ⊙ In particular, for a query term t_i :

$$u_i = p(x_i = 1 \mid \bar{R}, q) \approx \frac{\text{df}_{t_i}}{N}$$

- ⊙ Therefore:

$$\log \frac{1 - u_i}{u_i} \approx \log \frac{1 - \frac{\text{df}_{t_i}}{N}}{\frac{\text{df}_{t_i}}{N}} = \log \frac{N - \text{df}_{t_i}}{\text{df}_{t_i}}$$

- ⊙ If $\text{df}_{t_i} \ll N$, this is further approximated by:

$$\log \frac{N - \text{df}_{t_i}}{\text{df}_{t_i}} \approx \log \frac{N}{\text{df}_{t_i}}$$

Probability estimates in ad-hoc retrieval

- ⊙ In ad-hoc retrieval, no relevance judgements are available.
- ⊙ Therefore, $p_i = p(x_i = 1 \mid R, q)$ cannot be estimated from relevant documents.
- ⊙ A simple assumption is:

$$p_i = 0.5 \quad \text{for all query terms}$$

- ⊙ Recall that:

$$c_i = \log \frac{p_i(1 - u_i)}{u_i(1 - p_i)} = \log \frac{p_i}{1 - p_i} + \log \frac{1 - u_i}{u_i}$$

- ⊙ If $p_i = 0.5$, then:

$$\log \frac{p_i}{1 - p_i} = \log \frac{0.5}{0.5} = \log 1 = 0$$

- ⊙ Hence:

$$c_i = \log \frac{1 - u_i}{u_i}$$

From BIM to idf in ad-hoc retrieval

- From the previous approximation:

$$u_i \approx \frac{\text{df}_{t_i}}{N}$$

- Therefore:

$$c_i = \log \frac{1 - u_i}{u_i} \approx \log \frac{N - \text{df}_{t_i}}{\text{df}_{t_i}}$$

- If $\text{df}_{t_i} \ll N$, then:

$$c_i \approx \log \frac{N}{\text{df}_{t_i}}$$

- Thus the BIM score becomes:

$$RSV_d = \sum_{t_i : x_i=y_i=1} c_i \approx \sum_{t_i : x_i=y_i=1} \log \frac{N}{\text{df}_{t_i}}$$

- In the ad-hoc case, BIM therefore reduces to an idf-weighted score over query terms occurring in the document.

How different are vector space and BIM?

- ⊙ They are not that different.
- ⊙ In either case you build an information retrieval scheme in the exact same way.
- ⊙ For probabilistic IR, at the end, you score queries not by cosine similarity and tf-idf in a vector space, but by a slightly different formula motivated by probability theory.

Open issue: how to add term frequency and length normalization to the probabilistic model.

Key limitations of BIM

- ⊙ BIM, like much of original IR, was designed for titles or abstracts, and not for modern full text search
- ⊙ We want to pay attention to **term frequency** and **document lengths**
- ⊙ Want some model of how often terms occur in docs

Introducing term frequency

Let us first remind the definition of the Retrieval Status Value:

$$RSV_d = \log \prod_{t_i : x_i = y_i = 1} \frac{p(x_i = 1|R, v_q)p(x_i = 0|\bar{R}, v_q)}{p(x_i = 1|\bar{R}, v_q)p(x_i = 0|R, v_q)}$$

By still applying the simplifying assumption introduced for BIM, we approximate $p(x_i = 1|\bar{R}, v_q)$ by $p(x_i = 1)$ and $p(x_i = 0|\bar{R}, v_q)$ by $p(x_i = 0)$

$$RSV_d = \sum_{t_i : x_i = y_i = 1} \log \frac{p(x_i = 1|R, v_q)p(x_i = 0)}{p(x_i = 1)p(x_i = 0|R, v_q)}$$

From binary occurrence to term counts

In BIM, term t_i is modeled through a binary variable:

$$x_i \in \{0, 1\}$$

So we distinguish only two cases:

$$x_i = 1 \quad \text{or} \quad x_i = 0$$

To model term frequency, we replace x_i with the count variable

$$d_{t_i} = \text{\#occurrences of } t_i \text{ in } d$$

Hence the question changes from: $x_i \in \{0, 1\}$ to: $d_{t_i} \in \{0, 1, 2, \dots\}$

Why four probabilities?

With counts, the two events we care about are:

$$d_{t_i} = n_i \quad \text{and} \quad d_{t_i} = 0$$

Each event must be evaluated in two different contexts:

- ⊙ in relevant documents
- ⊙ in a baseline distribution, approximated by the whole collection

So we need four probabilities:

$$\begin{aligned} p(d_{t_i} = n_i \mid R, v_q), & \quad p(d_{t_i} = 0 \mid R, v_q) \\ p(d_{t_i} = n_i), & \quad p(d_{t_i} = 0) \end{aligned}$$

Therefore,

$$RSV_d = \sum_{t_i: y_i=1} \log \frac{p(d_{t_i} = n_i \mid R, v_q) p(d_{t_i} = 0)}{p(d_{t_i} = n_i) p(d_{t_i} = 0 \mid R, v_q)}$$

Modeling term occurrences

We want to model the random variable

d_{t_j} = number of occurrences of term t_j in document d

In other words, we want a probabilistic model for **term counts** in documents.

- ⊙ If a term is rare, we expect small values of d_{t_j}
- ⊙ If a term is central to the document topic, we may observe larger values
- ⊙ So the question is: which discrete distribution is suitable for modeling these counts?

A first possible answer is the **Binomial distribution**.

A first model: Binomial distribution

A simple abstraction is to view a document as made of l positions.

- ⊙ At each position, term t_j may occur with probability $\tilde{p}$
- ⊙ So the total number of occurrences of t_j in the document can be modeled as:

$$p(d_{t_j} = k) = \binom{l}{k} \tilde{p}^k (1 - \tilde{p})^{l-k}$$

- ⊙ Intuitively: we count how many times the term appears over many possible positions

This gives a reasonable first model, but it is often cumbersome to manipulate in practice.

Why Poisson?

A simpler alternative is the **Poisson distribution**.

- ⊙ Think of a document d as a sequence of l positions.
- ⊙ At each position, term t_j may occur with probability $\tilde{p}$.
- ⊙ Hence, the number of occurrences of t_j in d can first be modeled as:

$$d_{t_j} \sim \text{Binomial}(l, \tilde{p})$$

where:

- l is the document length;
- $\tilde{p}$ is the probability that one position contains t_j .
- ⊙ When l is large and $\tilde{p}$ is small,

$$\text{Binomial}(l, \tilde{p}) \approx \text{Poisson}(\lambda) \quad \text{with} \quad \lambda = l\tilde{p}$$

Thus, λ is the **expected number of occurrences** of term t_j in document d .

Poisson keeps the same intuition as the binomial model, but gives formulas that are simpler to estimate and manipulate.

Term occurrences as Poisson distribution

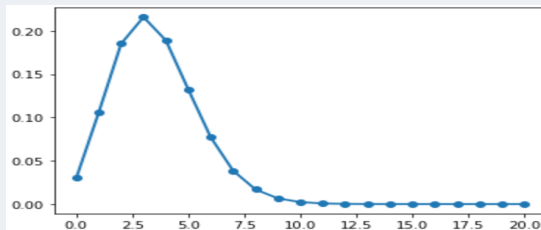
The **Poisson distribution** with mean parameter λ assigns probability

$$p(X = x) = \frac{e^{-\lambda} \lambda^x}{x!}$$

to observing exactly x occurrences of an event. In our setting, the event is the occurrence of term t_j in document d :

$$X = d_{t_j}$$

- ⊙ x is a possible number of occurrences of term t_j in document d
- ⊙ λ is the expected number of occurrences of t_j in document d



Term occurrences as Poisson distribution

How can we estimate λ ?

In the Poisson model, λ_j represents the average number of times term t_j occurs in a document.

A natural estimate is:

$$\lambda_j \approx \frac{cf_{t_j}}{N}$$

with cf_t the collection frequency of term t . That is, we take the total number of occurrences of t_j in the collection and divide it by the number of documents.

This gives the average number of occurrences of t_j per document, which is exactly what λ_j is meant to capture.

Term occurrences as Poisson distribution

We model the number of occurrences of a term t_j with a Poisson distribution.

Two different expected values are considered:

- ⊙ ρ_j : expected number of occurrences of t_j in documents relevant to query q
- ⊙ γ_j : expected number of occurrences of t_j in the collection as a whole

So:

- ⊙ ρ_j models the behavior of the term in relevant documents
- ⊙ γ_j models its average behavior in the whole collection

Term occurrences as Poisson distribution

Using these two parameters, we write:

$$p(d_{t_j} = n_j \mid R, v_q) = \frac{e^{-\rho_j} \rho_j^{n_j}}{n_j!}$$

$$p(d_{t_j} = 0 \mid R, v_q) = e^{-\rho_j}$$

$$p(d_{t_j} = n_j) = \frac{e^{-\gamma_j} \gamma_j^{n_j}}{n_j!}$$

$$p(d_{t_j} = 0) = e^{-\gamma_j}$$

So we compare the expected behavior of t_j in relevant documents with its expected behavior in the collection.

Term occurrences as Poisson distribution

As a consequence:

$$\begin{aligned} RSV_d &= \sum_{t_j: y_j=1} \log \frac{p(d_{t_j} = n_j \mid R, v_q) p(d_{t_j} = 0)}{p(d_{t_j} = n_j) p(d_{t_j} = 0 \mid R, v_q)} \\ &= \sum_{t_j: y_j=1} \log \frac{\frac{e^{-\rho_j} \rho_j^{n_j}}{n_j!} e^{-\gamma_j}}{\frac{e^{-\gamma_j} \gamma_j^{n_j}}{n_j!} e^{-\rho_j}} \\ &= \sum_{t_j: y_j=1} \log \frac{\rho_j^{n_j}}{\gamma_j^{n_j}} = \sum_{t_j: y_j=1} n_j \log \frac{\rho_j}{\gamma_j} \end{aligned}$$

Each occurrence of t_j contributes to the score by $\log(\rho_j/\gamma_j)$, that is, the log-ratio between its expected frequency in relevant documents and in the collection.

Can we estimate ρ_j and γ_j directly?

The collection-level parameter γ_j can be estimated directly from the collection:

$$\gamma_j \approx \frac{\sum_{d \in D} t f_{j,d}}{|D|} = \frac{c f_{t_j}}{N}$$

where $c f_{t_j}$ is the total number of occurrences of t_j in the collection.

In principle, if relevance judgements were available, we could also estimate:

$$\rho_j \approx \frac{\sum_{d \in R_q} t f_{j,d}}{|R_q|}$$

where R_q is the set of documents judged relevant for query q .

Can we estimate ρ_j and γ_j directly? (2)

However, in standard ad-hoc retrieval, R_q is unknown:

- ⊙ γ_j is observable from the collection;
- ⊙ ρ_j requires knowing relevant documents;
- ⊙ but relevant documents are exactly what we are trying to find.

So ρ_j cannot be reliably estimated directly in the first retrieval step.

From the ideal estimate to a practical score

The Poisson derivation is useful because it suggests the desired form of the score:

$$RSV_d = \sum_{t_j: y_j=1} n_j \log \frac{\rho_j}{\gamma_j}$$

But in ad-hoc retrieval:

- ⊙ γ_j can be estimated from the whole collection;
- ⊙ ρ_j would require knowing which documents are relevant;
- ⊙ therefore, the exact probabilistic score is not directly usable.

The next models keep the same intuition:

- ⊙ term frequency should increase the score;
- ⊙ rare terms should be more informative;
- ⊙ the effect of repeated occurrences should saturate.

- ⊙ The above assumptions fit rather well in the case of contentless terms, that is words which do not bear much meaning about a document topic
- ⊙ In the case of contentful terms, which may characterize with their occurrence the topic of a document, the situation may be different

Term occurrences as Poisson distribution

Table 1. Frequency Distributions for 19 Word Types and Expected Frequencies Assuming a Poisson Distribution with $\lambda = 53/650$

Frequency	Word Type	Number of Documents Containing k Tokens													
		k	0	1	2	3	4	5	6	7	8	9	10	11	12
51	act		608	35	5	2									
51	actions		617	27	2	0	2	0	2						
54	attitude		610	30	7	2	1								
52	based		600	48	2										
53	body		605	39	4	2									
52	castration		617	22	6	3	1	1							
55	cathexis		619	22	3	2	1	2	0	1					
51	comic		642	3	0	1	0	0	0	0	0	0	1	1	2
53	concerned		601	45	4										
53	conditions		604	39	7										
55	consists		602	41	7										
53	factor		609	32	7	1	1								
52	factors		611	27	11	1									
55	feeling		613	26	7	3	0	0	1						
52	find		602	45	2	1									
54	following		604	39	6	1									
51	force		603	43	4										
51	forces		609	33	6	2									
52	forgetting		629	11	3	2	2	1	1	0	0	0	1		
53	expected, assuming Poisson distribution		599	49	2										

Contentful words may have higher values of df_t : this happens for documents whose topic is described by the term

Two different Poisson distributions

Assume two types of terms occur in a document:

- ⊙ Terms which do not characterize the topic of the document
- ⊙ Terms which describe the topic of the document

Each class of terms is distributed according to a different Poisson: lower parameter for the first class, higher for the second class

The type of term in a document is modeled in terms of **eliteness**:

⊙ What is eliteness?

- **Hidden** binary variable for each document-term pair
- Given a document, a term is **elite** if, in some sense, the document is about the concept denoted by the term: this implies that such term will tend to appear more often in the document
- Term occurrences depend only on eliteness (not on relevance, at least directly)
- But eliteness is related to relevance

Term occurrences as 2-Poisson distribution

Let E_i denote the **elite** random variable for term t_i in the document considered. We assume that the distribution $p(d_{t_i} = n_i | R, v_q)$ can be expressed as the mixture of two Poisson distributions, for the elite and the not elite case.

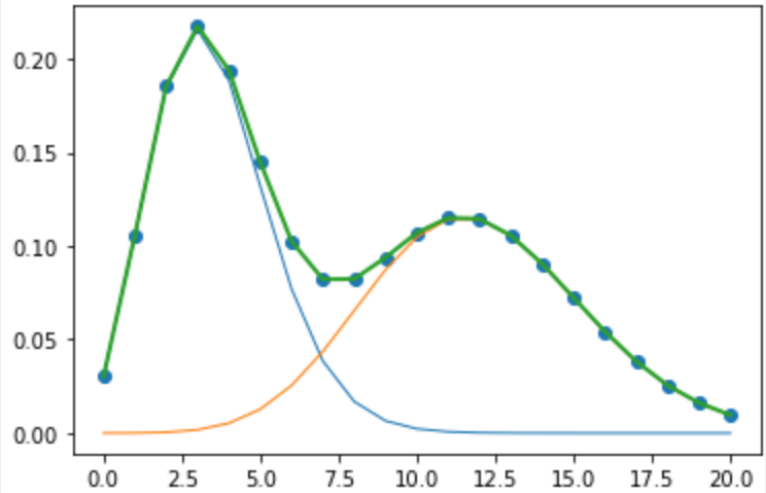
$$p(d_{t_i} = n_i | R, v_q) = p(d_{t_i} = n_i | E_i) p(E_i | R, v_q) + p(d_{t_i} = n_i | \bar{E}_i) p(\bar{E}_i | R, v_q)$$

$$p(d_{t_i} = n_i | R, v_q) = p_i \cdot \text{Poisson}(n_i | \mu_i) + (1 - p_i) \cdot \text{Poisson}(n_i | \bar{\mu}_i)$$

$$p(d_{t_i} = n_i | R, v_q) = p_i \frac{e^{-\mu_i} \mu_i^{n_i}}{n_i!} + (1 - p_i) \frac{e^{-\bar{\mu}_i} \bar{\mu}_i^{n_i}}{n_i!}$$

where $p_i = p(E_i | R, v_q)$ is the probability that the document is elite for the term t_i

Term occurrences as 2-Poisson distribution



Term occurrences as 2-Poisson distribution

The probabilities in RSV can be decomposed as

$$p(d_{t_i} = n_i | R, v_q) = C(n_i)p_i + \bar{C}(n_i)(1 - p_i)$$

$$p(d_{t_i} = 0 | R, v_q) = C(0)p_i + \bar{C}(0)(1 - p_i)$$

$$p(d_{t_i} = n_i) = C(n_i)\bar{p} + \bar{C}(n_i)(1 - \bar{p})$$

$$p(d_{t_i} = 0) = C(0)\bar{p} + \bar{C}(0)(1 - \bar{p})$$

where:

- ⊙ $C(n_i) = \text{Poisson}(n_i | \mu_i)$ is the probability of observing n_i occurrences of the term if the document is elite for it
- ⊙ $\bar{C}(n_i) = \text{Poisson}(n_i | \bar{\mu}_i)$ is the probability of observing n_i occurrences of the term if the document is not elite for it
- ⊙ $p_i = p(E_i | R, v_q)$ is the probability that the document is elite for t_i assuming it is relevant
- ⊙ $\bar{p} = p(E_i)$ is the probability that a document is elite for t_i in the collection as a whole, estimated by considering the whole collection

Where are we going?

We started from BIM, where a query term only contributes if it is present.

Now we want a non-binary contribution:

$$\text{term contribution} = \text{term-frequency evidence} \times \text{term importance}$$

The 2-Poisson model gives a probabilistic motivation for this idea:

- ⊙ term frequency estimates how likely the document is elite for the term
- ⊙ the binary probabilistic weight estimates how useful that elite status is for relevance

BM25 will keep this intuition, but replace the full probabilistic expression with a simpler robust approximation.

The full 2-Poisson score

For a single query term t_i , the 2-Poisson contribution is:

$$\log \frac{(C(n_i)p_i + \bar{C}(n_i)(1 - p_i)) (C(0)\bar{p} + \bar{C}(0)(1 - \bar{p}))}{(C(0)p_i + \bar{C}(0)(1 - p_i)) (C(n_i)\bar{p} + \bar{C}(n_i)(1 - \bar{p}))}$$

This formula is conceptually useful, but difficult to use directly.

It requires estimating, for each term:

- ⊙ μ_i : average count of t_i in elite documents
- ⊙ $\bar{\mu}_i$: average count of t_i in nonelite documents
- ⊙ $p_i = p(E_i = 1 \mid R, v_q)$ and $\bar{p} = p(E_i = 1)$

In practice, this is too expensive and unstable. See Robertson and Walker (1994), *Some Simple Effective Approximations to the 2-Poisson Model for Probabilistic Weighted Retrieval*.

https://www.staff.city.ac.uk/~sbrp622/papers/robertson_walker_sigir94.pdf

What behavior do we want?

Instead of using the full formula directly, we approximate its behavior.

For a term occurring n_i times, the contribution should:

1. be 0 when $n_i = 0$
2. increase when n_i increases
3. not grow without bound
4. saturate toward a bounded value proportional to a binary/idf-like weight

This captures the intuition:

more occurrences help, but after a while they add little new evidence.

A simple saturating approximation

A simple function with the desired behavior is:

$$\frac{(k+1)n_i}{k+n_i}$$

It has the right qualitative properties:

⊙ if $n_i = 0$, then

$$\frac{(k+1)n_i}{k+n_i} = 0$$

⊙ as n_i increases, the function increases

⊙ as $n_i \rightarrow \infty$, it saturates:

$$\frac{(k+1)n_i}{k+n_i} \rightarrow k+1$$

So term frequency is rewarded, but with diminishing returns.

Why diminishing returns?

Term frequency can be interpreted as evidence of **eliteness**.

- ⊙ If $n_i = 0$, there is no evidence that the document is about t_i .
- ⊙ If n_i becomes positive, the document becomes more likely to be elite for t_i .
- ⊙ But once n_i is large, the document is already very likely to be elite.

So additional occurrences should still help, but less and less:

$n_i = 1 \Rightarrow$ important evidence

$n_i = 10 \Rightarrow$ already strong evidence; one more occurrence changes little

This motivates a saturating term-frequency factor.

From eliteness to a binary probabilistic weight

So far, term frequency tells us **how much evidence** we have that a document is about term t_i .

But we also need to know **how important** that evidence is.

For this, we compare two probabilities:

- ⊙ $p_i = p(E_i \mid R, v_q)$: probability that a relevant document is elite for term t_i
- ⊙ $\bar{p} = p(E_i)$: probability that a generic document in the collection is elite for term t_i

If p_i is much larger than $\bar{p}$, then being elite for t_i is strong evidence of relevance.

The binary probabilistic weight measures this contrast.

Approximating the 2-Poisson contribution

The full 2-Poisson contribution is replaced by:

$$\frac{(k+1)n_i}{k+n_i} \cdot \log \frac{p_i(1-\bar{p})}{(1-p_i)\bar{p}}$$

The first factor models **term-frequency saturation**:

$$\frac{(k+1)n_i}{k+n_i}$$

The second factor is the **binary probabilistic weight**:

$$\log \frac{p_i(1-\bar{p})}{(1-p_i)\bar{p}}$$

So the approximation says: BM-style score = saturated term frequency $\times$ binary/idf-like weight

Ad-hoc retrieval: no elite information

In ad-hoc retrieval, we do not know:

- ⊙ which documents are relevant
- ⊙ which documents are elite for a term

So the probabilistic weight

$$\log \frac{p_i(1 - \bar{p})}{(1 - p_i)\bar{p}}$$

cannot be estimated directly.

We use two simple approximations:

$$p_i = p(E_i = 1 \mid R, v_q) \approx 0.5$$

$$\bar{p} = p(E_i = 1) \approx \frac{df_{t_i}}{N}$$

From eliteness weight to idf

Start from the binary probabilistic weight:

$$\log \frac{p_i(1 - \bar{p})}{(1 - p_i)\bar{p}} = \log \frac{p_i}{1 - p_i} + \log \frac{1 - \bar{p}}{\bar{p}}$$

If $p_i = 0.5$, then:

$$\log \frac{p_i}{1 - p_i} = 0$$

Thus:

$$\log \frac{p_i(1 - \bar{p})}{(1 - p_i)\bar{p}} \approx \log \frac{1 - \bar{p}}{\bar{p}}$$

Using $\bar{p} \approx df_{t_i}/N$:

$$\log \frac{1 - \bar{p}}{\bar{p}} \approx \log \frac{N - df_{t_i}}{df_{t_i}} \approx \log \frac{N}{df_{t_i}}$$

A first step toward BM25

Combining the two approximations:

saturated term frequency $\times$ idf-like weight

we obtain:

$$RSV_d = \sum_{t_i: y_i=1} \frac{(k+1)n_i}{k+n_i} \log \frac{N}{df_{t_i}}$$

This already contains two central ideas of BM25:

- ⊙ term frequency matters, but saturates
- ⊙ rare terms are more informative through idf

BM25 will further add **document length normalization**.

- ⊙ Okapi BM25 is a probabilistic model that incorporates term frequency (i.e., it's nonbinary) and length normalization.
- ⊙ BIM was originally designed for short catalog records of fairly consistent length, and it works reasonably in these contexts
- ⊙ For modern full-text search collections, a model should pay attention to term frequency and document length
- ⊙ BestMatch25 (a.k.a **BM25** or **Okapi**) is sensitive to these quantities
- ⊙ BM25 is one of the most widely used and robust retrieval models

From probabilistic modeling to BM25

The full probabilistic model is useful to understand the intuition, but it is too complex to estimate directly.

BM25 keeps the main qualitative consequences:

- ⊙ rare query terms should receive higher weight;
- ⊙ repeated occurrences should help, but with saturation;
- ⊙ document length should be normalized.

Thus, BM25 can be seen as a practical scoring function inspired by the probabilistic analysis.

- ⊙ The simplest score for document d is just idf weighting of the query terms present in the document:

$$RSV_d = \sum_{t \in q} \log \frac{N}{df_t}$$

Okapi BM25 first basic weighting

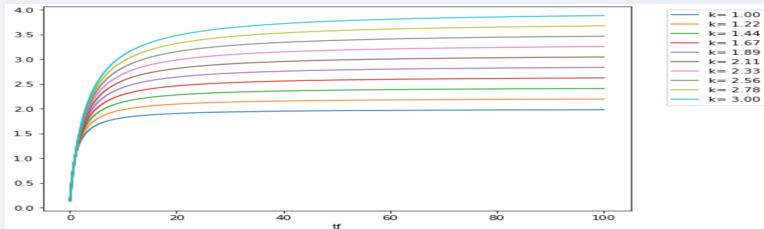
- ⊙ Improve idf term $\log \frac{N}{df_t}$ by factoring in term frequency.

$$RSV_d = \sum_{t \in q} \frac{(k_1 + 1)tf_{td}}{k_1 + tf_{td}} \log \frac{N}{df_t}$$

- ⊙ k_1 : tuning parameter controlling the document term frequency scaling
- ⊙ $(k_1 + 1)$ factor does not change ranking, but makes term score 1 when $tf_{td} = 1$
- ⊙ Similar to tf-idf, but term scores are bounded

Role of parameter k_1

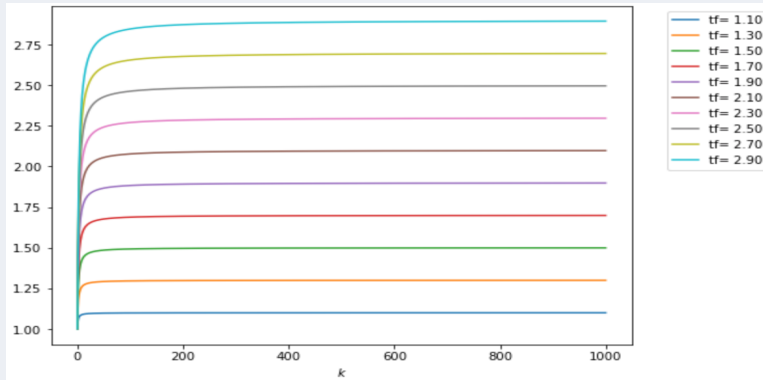
- ⊙ k_1 helps determine term frequency saturation characteristics
- ⊙ it limits how much a single query term can affect the score of a given document. It does this through approaching an asymptote



- ⊙ A higher/lower k_1 value means that the slope of tf of BM25 curve changes. This has the effect of changing how terms occurring extra times add extra score.
- ⊙ Usually, values around 1.2 – 2

Exercise

- ⊙ Interpret weighting formula for $k_1 = 0$
- ⊙ Interpret weighting formula for $k_1 = 1$
- ⊙ Interpret weighting formula for $k_1 \mapsto \infty$



Document length normalization

- ⊙ Longer documents are likely to have larger tf_{td} values
- ⊙ Why might documents be longer?
 - Verbosity: suggests observed tf_{td} too high
 - Larger scope: suggests observed tf_{td} may be right
- ⊙ A real document collection probably has both effects so we should apply some kind of partial normalization

Document length normalization

- ⊙ Document length

$$L_d = \sum_t \text{tf}_{td}$$

- ⊙ Document length average in the collection D

$$L_{\text{ave}} = \frac{1}{|D|} \sum_{d \in D} L_d$$

- ⊙ Length normalization component

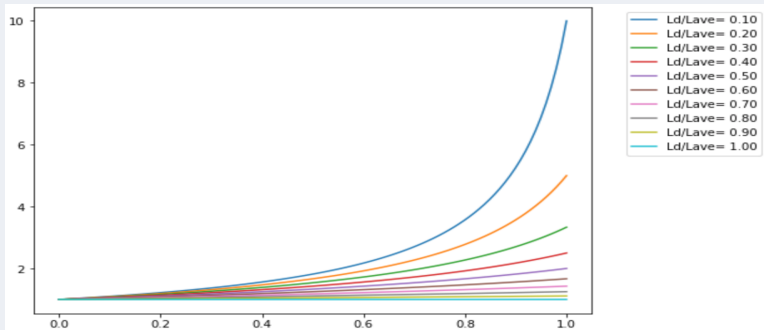
$$B = (1 - b) + b \frac{L_d}{L_{\text{ave}}} \qquad 0 \leq b \leq 1$$

- $b = 1$: full document length normalization
- $b = 0$: no document length normalization

Role of parameter b

The factor $B = (1 - b) + b \frac{L_d}{L_{ave}}$ appears in the denominator of the BM25 term-frequency component.

- ⊙ If $L_d > L_{ave}$, increasing b increases the normalization penalty.
- ⊙ If $L_d < L_{ave}$, increasing b decreases the normalization factor.
- ⊙ Therefore, larger b makes BM25 more sensitive to document length. Usually, b is set around 0.75



- ⊙ Improve idf term $\log \frac{N}{df_t}$ by factoring in term frequency and document length.

$$RSV_d = \sum_{t \in q} \frac{(k_1 + 1)tf_{td}}{k_1((1 - b) + b\frac{L_d}{L_{ave}}) + tf_{td}} \log \frac{N}{df_t}$$

- ⊙ tf_{td} : term frequency in document d
- ⊙ L_d (L_{ave}): length of document d (average document length in the whole collection)
- ⊙ k_1 : tuning parameter controlling the document term frequency scaling ($k_1 = 0$ is binary model, k_1 large is raw term frequency); usually around 1.2-2
- ⊙ b : tuning parameter controlling the scaling by document length ($b = 0$ is no normalization, $b = 1$ is full normalization); usually around .75

Exercise

- ⊙ Interpret BM25 weighting formula for $k_1 = 0$
- ⊙ Interpret BM25 weighting formula for $k_1 = 1$ and $b = 0$
- ⊙ Interpret BM25 weighting formula for $k_1 \mapsto \infty$ and $b = 0$
- ⊙ Interpret BM25 weighting formula for $k_1 \mapsto \infty$ and $b = 1$

- ⊙ Suppose your query is [machine learning]
- ⊙ Suppose you have 2 documents with term counts:
 - doc1: learning 1024; machine 1
 - doc2: learning 16; machine 8
- ⊙ Suppose that machine occurs in 1 out of 7 documents in the collection
- ⊙ Suppose that learning occurs in 1 out of 10 documents in the collection
- ⊙ tf-idf: $1 + \log_{10}(1 + tf) \log_{10}(N/df)$
 - doc1: 41.1
 - doc2: 35.8
- ⊙ BM25: $k_1 = 2$
 - doc1: 31
 - doc2: 42.6

Okapi BM25 weighting for long queries

- ⊙ For long queries, use similar weighting for query terms

$$RSV_d = \sum_{t \in q} \left[\log \frac{N}{df_t} \right] \cdot \frac{(k_1 + 1)tf_{td}}{k_1((1 - b) + b \times (L_d/L_{ave})) + tf_{td}} \cdot \frac{(k_3 + 1)tf_{tq}}{k_3 + tf_{tq}}$$

- ⊙ tf_{tq} : term frequency in the query q
- ⊙ k_3 : tuning parameter controlling term frequency scaling of the query
- ⊙ No length normalization of queries (because retrieval is being done with respect to a single fixed query)
- ⊙ The above tuning parameters should ideally be set to optimize performance on a development test collection. In the absence of such optimization, experiments have shown reasonable values are to set k_1 and k_3 to a value between 1.2 and 2 and $b = 0.75$

Which ranking model should I use?

- ⊙ I want something basic and simple → use vector space with tf-idf weighting.
- ⊙ I want a strong, robust first-stage ranking model → use BM25 (or language models) with **tuned parameters**
- ⊙ In between: BM25 or language models with no or just one tuned parameter

BM25 is not only theoretically motivated: it is also widely used in real search engines.

- ⊙ Elasticsearch uses BM25 as its default similarity model.
- ⊙ Apache Solr uses BM25 as its default similarity since Solr 8.x.

Elasticsearch documentation:

<https://www.elastic.co/docs/reference/elasticsearch/index-settings/similarity>

Solr documentation: <https://solr.apache.org/guide/solr/latest/indexing-guide/field-type-definitions-and-properties.html>